第 2 讲：数组、Python `list` 与原地修改

0. 本讲的问题

假设有一个列表：

```
[python]
nums = [3, 2, 2, 3, 4]
```

现在要求删除所有的 `3`，但有两个限制：

1. 不要创建一个与输入同样长的新列表；
2. 函数需要直接修改原列表的前半部分。

这不是单纯的“会不会删除元素”。它要求我们回答：

> 当从左向右扫描时，怎样让已经处理过的区间始终保持正确？

这个问题会引出本讲最重要的编码工具：**读指针、写指针和循环不变量**。

---

1. 数组是什么

从抽象层面看，数组是一串按顺序排列、可以通过整数下标访问的槽位：

```
  0   1   2   3
[8] [5] [9] [2]
```

访问 `nums[2]` 时，程序不需要先查看 `nums[0]` 和 `nums[1]`，因此按下标读取或修改通常是 `O(1)`：

```
[python]
x = nums[2]     #
nums[2] = 100   #
```

1.1 Python `list` 更准确地说是动态数组

C 的普通数组通常具有固定长度。Python 的 `list` 可以执行：

```
[python]
nums.append(7)
```

长度能够增长，所以它更接近“动态数组”。

从实现直觉上，可以把 Python `list` 想成一排保存对象引用的槽位。它不是 C 中那种直接紧密存放同类型整数的普通数组，因此：

- 元素可以是不同类型；
- 列表保存的是对象引用；
- 列表自身可以扩容；
- 访问第 `i` 个槽位仍然是 `O(1)`。

本课程中说“数组题”时，通常使用 Python `list` 来完成。

---

2. 为什么 append 平均是 O(1)

如果列表后面还有预留空间，append 只需要把新元素放进下一个槽位，因此是 O(1)。

如果空间用完，Python 需要申请更大的区域，并复制已有引用。某一次扩容可能是 O(n)。

但它不会每添加一个元素就扩容一次，而是一次多申请一些容量。假设容量大致按比例增长，连续添加 n 个元素时，复制量类似：

$$1+2+4+8+\cdots < 2n$$

因此，n 次 append 的总成本仍然是 O(n)，平均到每一次就是 O(1)。这种分析称为**摊还复杂度**。

你不需要记住 Python 的实际扩容比例，只要记住：

> list.append 是摊还 O(1)，偶尔一次扩容会更慢。

---

3. 常见操作的复杂度

设列表长度为 n：

操作	常见复杂度	原因
nums[i]	O(1)	直接定位槽位
nums[i] = x	O(1)	直接修改槽位
len(nums)	O(1)	长度被单独保存
nums.append(x)	摊还 O(1)	通常在末尾直接加入
nums.pop()	O(1)	删除末尾元素
nums.insert(0, x)	O(n)	后面的元素需要移动
nums.pop(0)	O(n)	后面的元素需要前移
x in nums	O(n)	最坏需要逐个比较
nums.index(x)	O(n)	最坏需要逐个查找
nums[a:b]	O(b-a)	创建并复制切片
nums.copy()	O(n)	创建浅拷贝
nums.sort()	O(n log n)	对原列表排序
sorted(nums)	O(n log n)	创建新列表并排序

“原地排序”和“返回新列表”不同：

```[python]
nums.sort()           #    nums    None
result = sorted(nums)  # nums      result
```

---

4. 什么时候遍历值，什么时候遍历下标

4.1 只需要元素值

```
[python]
for x in nums:
    print(x)
```

适用于求和、计数、查找等。

4.2 需要下标

```
[python]
for i in range(len(nums)):
    nums[i] *= 2
```

适用于修改原列表、访问相邻元素、维护多个位置。

4.3 同时需要下标和值

```
[python]
for i, x in enumerate(nums):
    print(i, x)
```

不要为了显得简洁而强行选择某种写法。应根据算法需要选择。

---

5. 下标区间：编码错误的高发区

Python 常用左闭右开区间：

```
[left, right)
```

意思是包含 `left`，不包含 `right`。

```
[python]
range(2, 5)   #    2, 3, 4
nums[2:5]     #      2, 3, 4
```

左闭右开的好处是区间长度直接等于：

\[right-left \]

5.1 常用范围

```
[python]
range(len(nums))              # 0   n-1
range(1, len(nums))           # 1   n-1
range(len(nums) - 1)          # 0   n-2
range(len(nums) - 1, -1, -1)  # n-1   0
```

5.2 相邻元素

如果代码访问 `nums[i + 1]`，就必须保证 `i + 1 < len(nums)`：

```
[python]
for i in range(len(nums) - 1):
    print(nums[i], nums[i + 1])
```

每当代码出现 `i-1` 或 `i+1`，都要重新检查循环边界。

---

6. 切片不是免费的

```
[python]
part = nums[1:4]
```

`part` 是一个新列表，元素来自原列表的下标 `1 2 3`。假设切片长度为 `k`：

- 时间复杂度：`O(k)`；
- 额外空间复杂度：`O(k)`。

所以：

```
[python]
for x in nums[1:]:
    ...
```

虽然循环思想可能只需要 `O(1)` 额外状态，但 `nums[1:]` 会额外创建长度约为 `n` 的列表。

如果需要严格的 `O(1)` 额外空间，可以写：

```
[python]
for i in range(1, len(nums)):
    x = nums[i]
```

切片非常实用，不是不能使用；重点是知道它会复制。

---

7. 引用、别名与浅拷贝

观察：

```
[python]
a = [1, 2, 3]
b = a
b[0] = 100

print(a)  # [100, 2, 3]
```

`a` 和 `b` 指向同一个列表。`b = a` 没有复制列表，只是增加了一个名字。

如果希望得到一个新的外层列表：

```
[python]
b = a.copy()
#
b = a[:]
```

但两者都是浅拷贝。对于嵌套列表：

```
[python]
a = [[1, 2], [3, 4]]
b = a.copy()
b[0][0] = 99

print(a)  # [[99, 2], [3, 4]]
```

外层列表不同，但内部的子列表仍然是共享对象。

7.1 一个经典陷阱

```
[python]
grid = [[0] * 3] * 3
grid[0][0] = 1
```

结果是：

```
[python]
```

```
[[1, 0, 0],
 [1, 0, 0],
 [1, 0, 0]]
```

因为三个位置引用的是同一个内部列表。

正确创建独立行：

```
[python]
grid = [[0] * 3 for _ in range(3)]
```

---

8. 原地修改是什么意思

原地修改通常表示：

> 直接改变输入对象，而不是返回一个替代它的新对象。

下面不是原地修改调用者持有的列表：

```
[python]
def clear_numbers(nums):
    nums = []
```

这里只是让函数内部的名字 `nums` 指向新列表，外面的原列表没有改变。

下面会修改原列表：

```
[python]
def clear_numbers(nums):
    nums.clear()
```

切片赋值也会保留原列表对象：

```
[python]
nums[:] = [1, 2, 3]
```

这在题目要求“修改输入数组”时很重要。

---

9. 核心推导：删除指定值

问题：把所有不等于 `val` 的元素移动到列表前面，并返回有效元素数量 `k`。

例如：

```
nums = [3, 2, 2, 3, 4], val = 3
```

最后只要求：

```
k = 3
nums[:k] == [2, 2, 4]
```

`k` 之后的内容不重要。

9.1 暴力想法的问题

可以反复执行 `nums.pop(i)`，但中间删除会让后面的元素移动，而且遍历过程中列表长度发生变化，很容易跳过元素。最坏时间也可能达到 `O(n²)`。

9.2 将问题改写为“保留谁”

与其删除 `val`，不如把所有不等于 `val` 的元素依次写到前面。

使用两个下标：

- `read`：当前正在检查的位置；
- `write`：下一个应该写入有效元素的位置。

循环开始时维持：

> `nums[0:write]` 恰好包含此前读到的所有有效元素，并保持原顺序。

代码：

```
[python]
def remove_value(nums, val):
    write = 0

    for read in range(len(nums)):
        if nums[read] != val:
            nums[write] = nums[read]
            write += 1

    return write
```

9.3 手动执行

输入：

```
nums = [3, 2, 2, 3, 4], val = 3
```

read	当前值	是否保留	写入位置	更新后 write	已确定区间
0	3	否	—	0	`[]`
1	2	是	0	1	`[2]`
2	2	是	1	2	`[2, 2]`
3	3	否	—	2	`[2, 2]`
4	4	是	2	3	`[2, 2, 4]`

结束时，所有元素都已读取，根据不变量，`nums[:write]` 就是全部有效元素，所以返回 `write`。

9.4 复杂度

- 每个元素读取一次：`O(n)` 时间；
- 只增加固定数量变量：`O(1)` 额外空间。

这里的两个指针并不是真正的 C 指针，只是两个保存下标的整数变量。

---

10. 为什么不应边遍历边删除

危险的写法：

```
[python]
nums = [1, 2, 2, 3]

for x in nums:
    if x == 2:
        nums.remove(x)
```

删除第一个 `2` 后，后一个 `2` 会向前移动，但循环的内部位置继续前进，于是可能跳过它。

如果目的只是创建过滤后的新列表，可以写：

```
[python]
result = []

for x in nums:
    if x != 2:
        result.append(x)
```

如果要求原地操作，使用读写指针更可靠。

---

11. 第二个编码模型：从两端向中间

反转列表时，可以交换两端：

```
[1, 2, 3, 4, 5]
 ↑           ↑
left       right
```

交换后，同时向中间移动：

```
[python]
def reverse_in_place(nums):
    left = 0
    right = len(nums) - 1

    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1
```

循环不变量可以描述为：

> `left` 左边和 `right` 右边的元素，已经位于反转后的正确位置。

每次循环至少确定两个位置，因此时间复杂度是 `O(n)`，额外空间是 `O(1)`。

---

12. 写数组代码前的检查表

```
1.
2.
3.
4.    i+1   i-1
5.
6.
7.           O(n)
8.
9.
10.
```

---

13. 本讲总结

1. Python `list` 是动态数组，支持 `O(1)` 下标访问。
2. 末尾添加是摊还 `O(1)`，头部或中间插入、删除通常是 `O(n)`。
3. 切片会创建新列表，不是免费的。
4. `b = a` 只创建别名；`a.copy()` 创建浅拷贝。

5. 原地修改要求改变原对象，而不是只让局部变量指向新对象。
6. 读写指针适合稳定地筛选、覆盖和压缩数组。
7. 两端指针适合反转或利用有序性缩小搜索范围。
8. 写代码时要明确循环不变量：数组的哪一段已经满足什么性质。

接下来打开 Notebook 练习。核心题是“移动零”和“有序数组的平方”，它们会要求你把本讲的两个编码模型迁移到新问题中。